

# SQL vs NoSQL Comparative Report

E-Commerce Order Management Database System | Skill Development Activity (DBMS)

## 1. Objective

Compare relational (SQL) and NoSQL databases with respect to schema, scalability, consistency, performance and use cases, and explain why organizations use different database technologies for different requirements. The comparison is grounded in the e-commerce project: the relational design from Week 6 and the NoSQL product catalog from Week 7.

## 2. Overview of the Two Approaches

**SQL (relational) databases** store data in tables of rows and columns with a predefined schema. Relationships are expressed through primary and foreign keys and queried with joins. They follow the ACID properties (Atomicity, Consistency, Isolation, Durability). Examples: MySQL, PostgreSQL, Oracle, SQL Server.

**NoSQL databases** ("not only SQL") store data in non-tabular models and usually favour flexibility and horizontal scale. They come in four main families:

| Type          | How data is stored              | Examples           | E-commerce use                  |
|---------------|---------------------------------|--------------------|---------------------------------|
| Document      | JSON-like documents             | MongoDB, Couchbase | Product catalog, reviews        |
| Key-value     | Key mapped to a value           | Redis, DynamoDB    | Carts, sessions, stock counters |
| Column-family | Wide columns grouped by row key | Cassandra, HBase   | Clickstream, activity logs      |
| Graph         | Nodes and edges                 | Neo4j              | Recommendations, fraud rings    |

## 3. Feature-by-Feature Comparison

| Feature     | SQL                                                                 | NoSQL                                |
|-------------|---------------------------------------------------------------------|--------------------------------------|
| Schema      | Fixed, defined before data is stored                                | Flexible, can differ per record      |
| Data model  | Tables, rows, columns                                               | Documents, key-value, column, graph  |
| Scalability | Mainly vertical (bigger server); horizontal scaling is possible but | Mainly horizontal (add more servers) |

| Feature        | SQL                                             | NoSQL                                                   |
|----------------|-------------------------------------------------|---------------------------------------------------------|
|                | harder                                          |                                                         |
| Consistency    | Strong (ACID)                                   | Often eventual (BASE), tunable in many systems          |
| Query language | Standard SQL                                    | Database-specific APIs or query languages               |
| Joins          | Powerful, built in                              | Limited or avoided; data is embedded instead            |
| Transactions   | Mature multi-row, multi-table transactions      | Single-record by default; multi-document support varies |
| Data integrity | Enforced by constraints (PK, FK, CHECK, UNIQUE) | Mostly enforced by the application                      |
| Normalization  | Normalized to reduce redundancy                 | Often denormalized for read speed                       |
| Best with      | Structured, related data                        | Semi-structured, varied or very large data              |
| Examples       | MySQL, Oracle, PostgreSQL                       | MongoDB, Redis, Cassandra                               |

## 4. Key Concepts Explained

### 4.1 Schema: Fixed vs Flexible

In SQL, adding a new product attribute means altering the table. A phone's `battery_mah` would be a NULL column for every T-shirt. In a document database, each product simply carries the attributes it needs, as shown in the Week 7 catalog design. The trade-off is that the database no longer guarantees every product has the same shape, so the application must validate it.

### 4.2 Scalability: Vertical vs Horizontal

Vertical scaling upgrades one machine (more CPU, RAM, storage). It is simple but has a hardware ceiling and a single point of failure. Horizontal scaling spreads data across many commodity servers (sharding). Most NoSQL systems were designed for this from the start, which is why they are common in very high-volume applications.

### 4.3 Consistency: ACID vs BASE

ACID guarantees suit money and stock:

- **Atomicity:** a transaction fully succeeds or fully fails
- **Consistency:** constraints are never violated
- **Isolation:** concurrent transactions do not interfere

- **Durability:** committed data survives failures

Many NoSQL systems follow BASE (Basically Available, Soft state, Eventually consistent). Replicas may briefly disagree but converge, which gives higher availability and speed. This relates to the CAP theorem: when a network partition occurs, a distributed system must choose between consistency and availability.

#### 4.4 Joins vs Embedding

A relational product page may need joins across product, category, supplier and review tables. A document design embeds category and supplier summaries inside the product so one read returns everything. Reads get faster, but updates to shared data (such as a category rename) must be applied in many places.

### 5. Advantages

#### 5.1 Advantages of SQL

1. **Strong integrity.** Primary keys, foreign keys, unique and check constraints keep data correct at the database level.
2. **Reliable transactions.** ACID guarantees protect payments, stock updates and order placement.
3. **Powerful querying.** Standard SQL supports joins, aggregation, subqueries and ad-hoc analytics.
4. **Mature ecosystem.** Decades of tooling, skills, security features and reporting support.
5. **Low redundancy.** Normalization avoids insertion, deletion and update anomalies.
6. **Procedural logic.** PL/SQL procedures, functions and triggers enforce business rules inside the database.

#### 5.2 Advantages of NoSQL

1. **Flexible schema.** New attributes or product types need no migration.
2. **Horizontal scalability.** Capacity grows by adding nodes.
3. **Fast reads for known access patterns.** One document or key lookup replaces multiple joins.
4. **High availability.** Replication across nodes and regions tolerates failures.
5. **Natural fit for varied data.** JSON maps directly to application objects.
6. **Specialised models.** Graph, key-value and column stores are efficient for their specific workloads.

### 6. Limitations

| SQL limitations                                           | NoSQL limitations                                  |
|-----------------------------------------------------------|----------------------------------------------------|
| Schema changes on large tables can be slow and disruptive | Weaker or eventual consistency can show stale data |
| Horizontal scaling is complex                             | Data duplication makes updates harder              |
| Rigid structure suits varied data poorly                  | Limited joins and ad-hoc queries                   |
| Heavy joins can slow very large read workloads            | Integrity rules mostly live in application code    |

| SQL limitations | NoSQL limitations                                             |
|-----------------|---------------------------------------------------------------|
|                 | Less standardisation; skills do not transfer between products |

## 7. Use Cases

### 7.1 When to Choose SQL

- Banking, payments and accounting
- Order management and inventory source of truth
- ERP, HR and CRM systems
- Reporting and business analytics with complex joins
- Any system where data relationships and correctness are the priority

### 7.2 When to Choose NoSQL

- Product catalogs with varied attributes
- Shopping carts and user sessions
- Real-time analytics and event or log data
- Content management and user profiles
- Social networks and recommendation engines (graph)
- IoT and sensor data at large volume

### 7.3 Use Cases in the E-Commerce Project

| Project component                 | Best fit          | Reason                                                   |
|-----------------------------------|-------------------|----------------------------------------------------------|
| Customers, addresses              | SQL               | Structured, relational, needs uniqueness (email, mobile) |
| Orders and order details          | SQL               | Multi-table transaction must be all-or-nothing           |
| Payments                          | SQL               | Money requires ACID guarantees and auditability          |
| Inventory stock (source of truth) | SQL               | Must never oversell; consistency matters                 |
| Stock counters for fast checks    | NoSQL (key-value) | Atomic counters and very low latency                     |
| Product catalog                   | NoSQL (document)  | Varied attributes, read-heavy, one read per page         |
| Reviews                           | NoSQL (document)  | Grows without limit, flexible content                    |

| Project component           | Best fit          | Reason                                                        |
|-----------------------------|-------------------|---------------------------------------------------------------|
| Cart and sessions           | NoSQL (key-value) | Temporary data with expiry, very fast                         |
| Product recommendations     | NoSQL (graph)     | Relationship traversal (customers who bought X also bought Y) |
| Sales reports and analytics | SQL               | Aggregations and joins across many tables                     |

## 8. Why Organizations Use Different Databases

No single database is best at everything. A transaction needs correctness, a catalog needs flexibility, and a session store needs speed. Large e-commerce companies therefore use **polyglot persistence**: several database technologies, each chosen for the workload it handles best.

Amazon is a well-known example. Its published Dynamo design, which led to the DynamoDB service, was driven by the need for an always-available shopping cart that keeps working during failures, accepting eventual consistency in return. Order and payment processing, by contrast, depends on strong transactional guarantees. Companies at this scale typically combine relational databases for transactions, key-value or document stores for catalogs and carts, search engines for product search, and data warehouses for analytics.

## 9. Recommended Architecture for the Project

|                                               |                                 |
|-----------------------------------------------|---------------------------------|
| Customer, Order, Payment, Shipment, Inventory | --> SQL database (MySQL/Oracle) |
| Product catalog, Reviews                      | --> Document store (MongoDB)    |
| Cart, Sessions, Stock cache                   | --> Key-value store (Redis)     |
| Sales reports                                 | --> SQL queries (Weeks 12-13)   |

The SQL database remains the source of truth for money and stock. NoSQL stores serve fast, flexible, read-heavy data. Cached or duplicated values in NoSQL must be synchronised with the SQL source.

## 10. Conclusion

SQL and NoSQL are complementary, not competing. SQL offers integrity, transactions and powerful querying and is the right foundation for orders, payments and inventory. NoSQL offers flexibility, horizontal scale and speed and is the right choice for catalogs, carts and high-volume data. The best design for a modern e-commerce platform uses each where its strengths matter most.